[Dat Nguyen] Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

1. ARM BOOT PROCESS: ROM CODE AND BOOTLOADER

Suppose you have an Embedded Linux board that uses an ARM processor.

Before power is applied, the CPU is not executing the operating system.

After power is applied and the hardware enters its startup state, the CPU begins executing code according to the boot rules designed for that SoC.

A natural question arises:

If the Linux kernel has not started yet, what program controls the CPU and loads the Linux kernel?

This is why we need to understand Boot ROM and the bootloader.

1.1 What is Boot ROM (ROM code)?

Boot ROM (read-only memory) is a memory region containing the initial boot code. This code is permanently programmed by the manufacturer into read-only memory integrated into the SoC.

- **Executed first:** This is the first code that runs when the device is powered on or after a hard reset. When the hardware is first powered on, it starts from a minimal state: RAM is empty, and the CPU does not yet know where to read the next instructions. Boot ROM is a small piece of code permanently built into the SoC chip during manufacturing. The CPU is designed to jump automatically to the Boot ROM address to fetch its first instructions.
- **Cannot be changed:** Because it resides in fixed ROM/Mask ROM, this code cannot normally be overwritten or modified by the user during operation.

- **Basic functions:**
  - Initialize the minimum required hardware, such as registers, clocks, and timers.
  - Check and configure the boot-mode pins.
  - Locate, authenticate (Secure Boot), and load the next boot program (the First-Stage Bootloader) from external storage, such as eMMC, an SD card, flash memory, or a serial interface, into RAM so execution can continue.

After completing the basic setup, the Boot ROM locates and authenticates the bootloader as required by Secure Boot, loads it from storage into RAM, and transfers control to it.

1.2 What is a bootloader?

A bootloader is a program that runs during the boot process and is responsible for preparing and loading the operating system.

- **Hardware initialization:** Checks and initializes components such as the CPU, RAM, and storage before the operating system starts.
- **Security and verification:** Verifies the integrity of the operating system to prevent malicious code or counterfeit firmware from running.

Why do we need a bootloader?

- The Linux kernel is much larger than the initial boot code.
- The system must prepare certain conditions before the kernel can run.
- The bootloader performs the preparation required before starting the kernel.

2. U-Boot

U-Boot stands for Universal Boot Loader.

U-Boot is widely used on embedded Linux boards, including many ARM-based boards.

It is a widely used open-source bootloader for embedded systems.

U-Boot's strengths come from the design principles its developers set out:

- Keep it Small
- Keep it Fast
- Keep it Simple
- Keep it Configurable
- Keep it Debuggable
- Keep it Usable
- Keep it Maintainable

- Keep it Beautiful
- Keep it Open

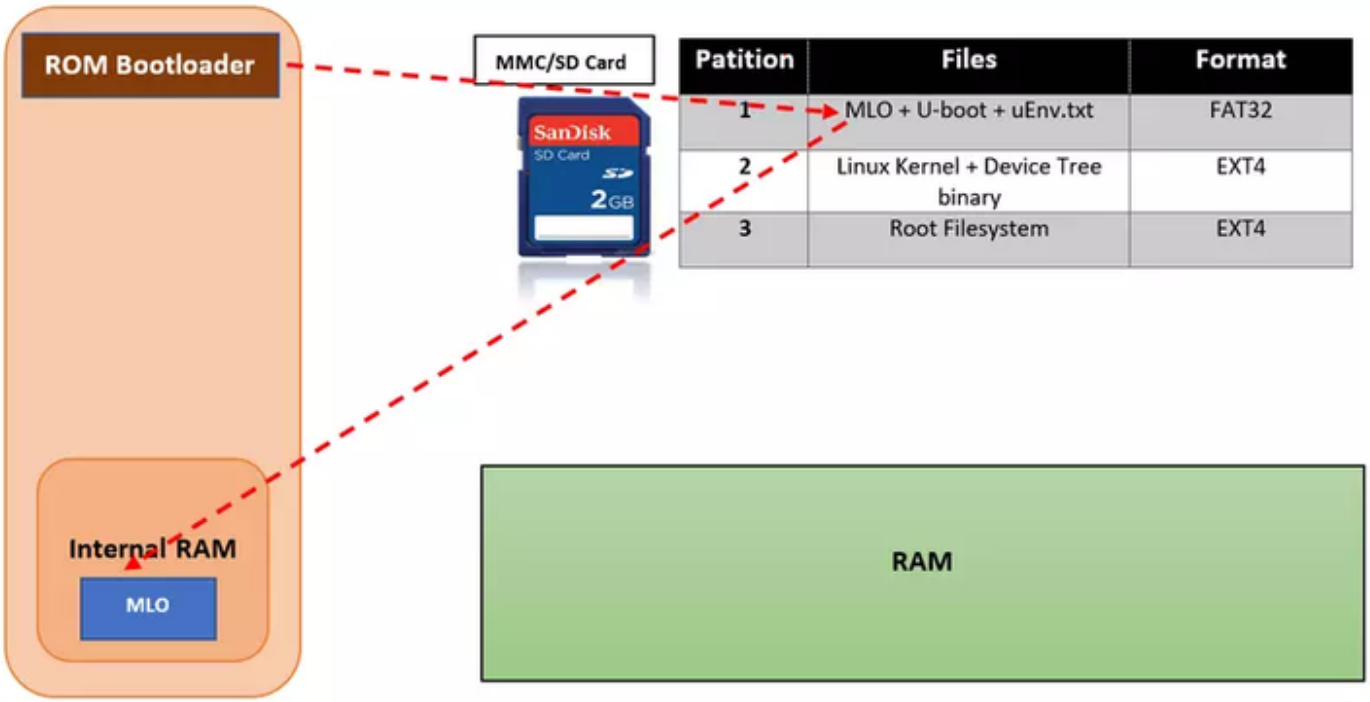
SoC ROM Bootloader

When the system is powered on for the first time, or reset, control begins at the reset vector: a location defined by the chip manufacturer that directs execution to the initial boot code. The reset vector points to the memory address containing the first boot code, namely the Boot ROM. Without a reset vector, the processor would not know where to start executing.

The main job of the Boot ROM is to copy the contents of the `MLO` file, also called the Second Program Loader (SPL), into internal RAM (IRAM) and execute it.

Because Boot ROM is quite small, its code is limited to initializing only the hardware needed to load the SPL, such as MMC/eMMC, an SD card, or NAND flash. These are collectively called the boot device.

Example: loading from an SD card:

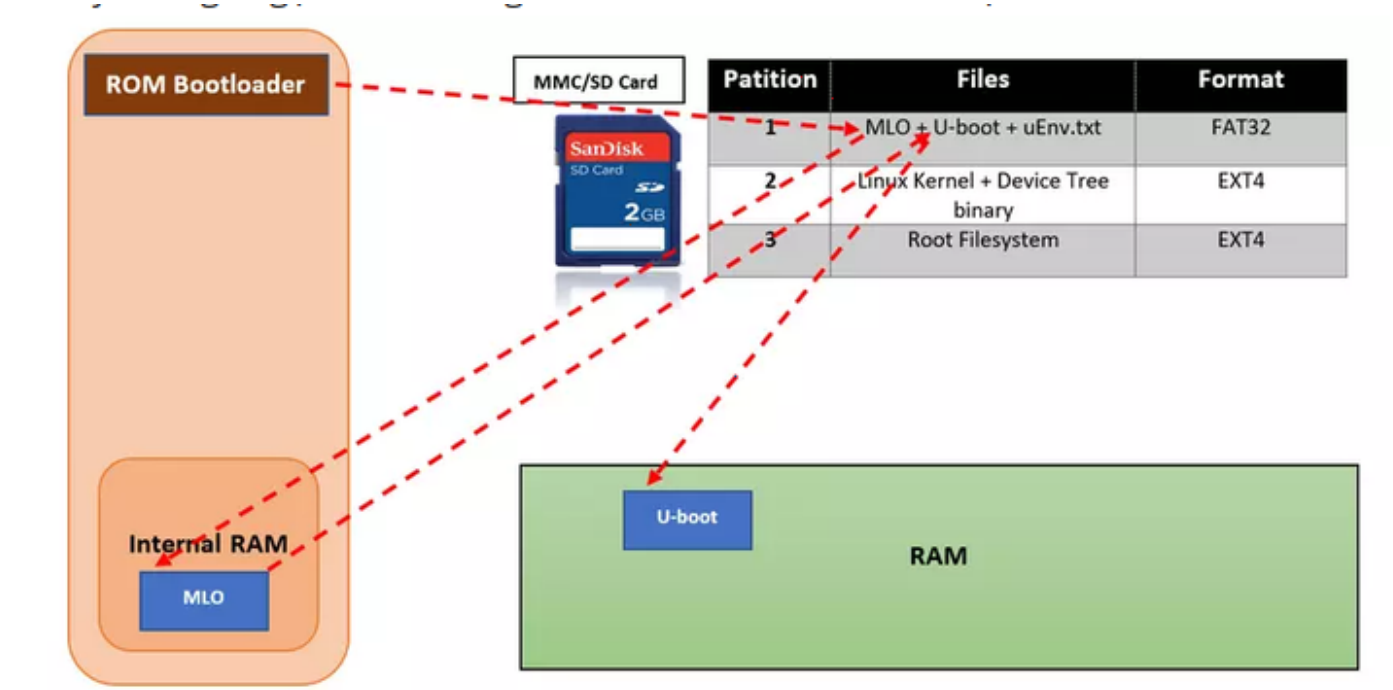


The boot device selected by the ROM code (SD card, flash, etc.) depends on how the boot pins are configured using switches or jumpers on the hardware.

Second Program Loader (SPL)

The SPL is a secondary loader. Its main job is to initialize additional components, such as the DRAM controller and eMMC, and then load U-Boot into RAM at the address specified by CONFIG_SYS_TEXT_BASE.

In short, the SPL's purpose is to get U-Boot into RAM.

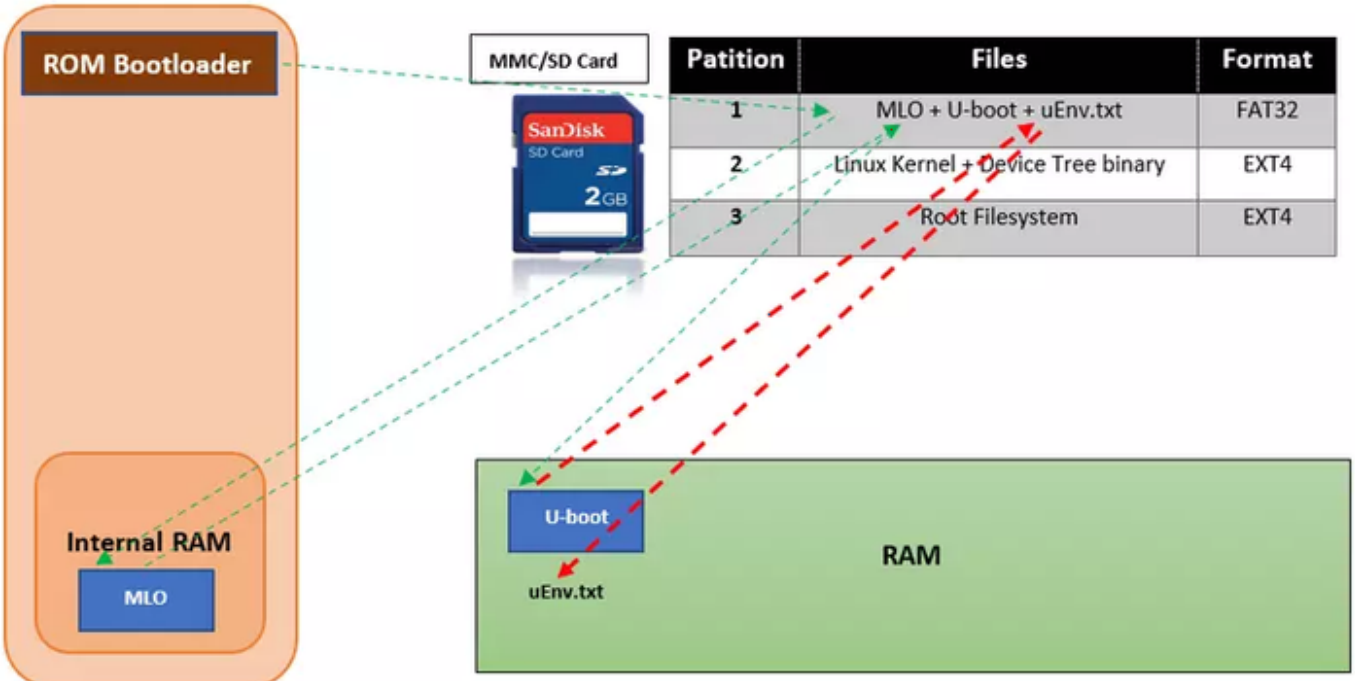


U-Boot

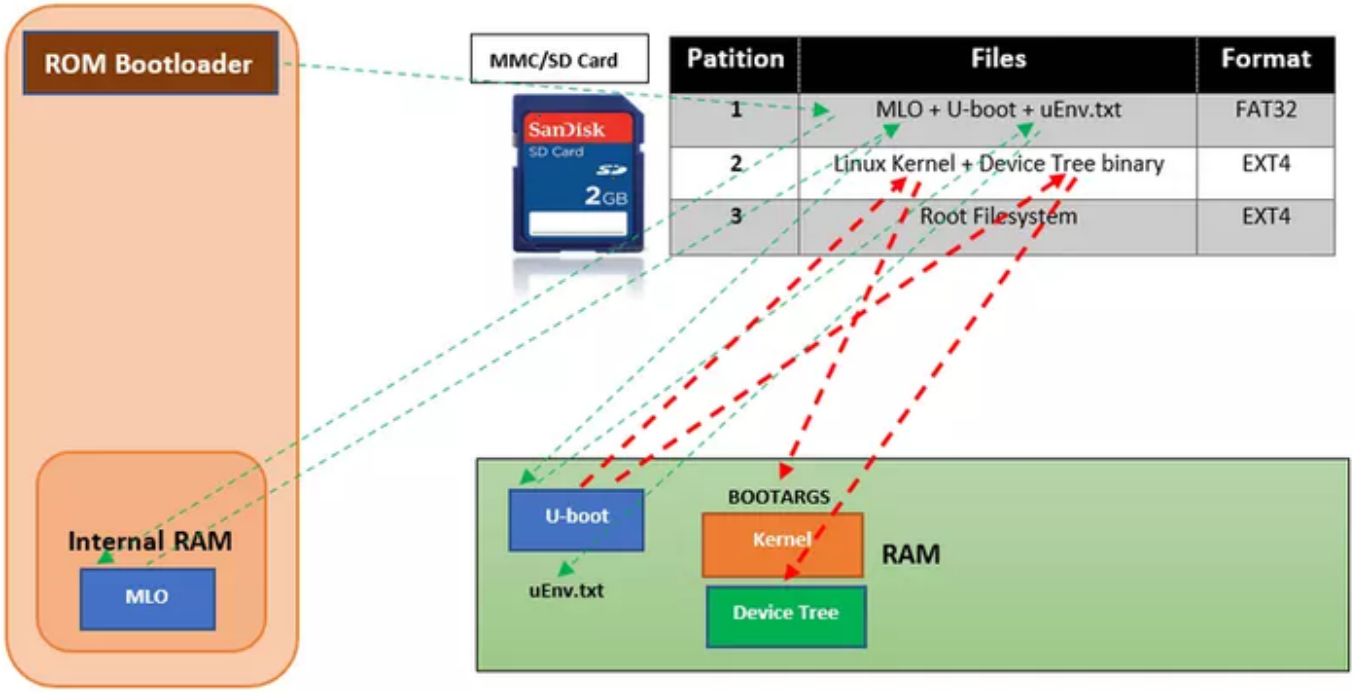
After being loaded into RAM, U-Boot performs relocation: it moves itself to the `relocaddr` address in RAM (usually near the end of RAM) and jumps to the relocated code.

U-Boot then checks whether a `uEnv.txt` file exists. If it does, U-Boot loads it into RAM in the next step.

uEnv.txt is a boot configuration file that defines configuration parameters and kernel command-line parameters. These parameters have defaults configured in U-Boot, but they can be added, modified, or removed through uEnv.txt. Loading uEnv.txt is optional, so the file may or may not be present.

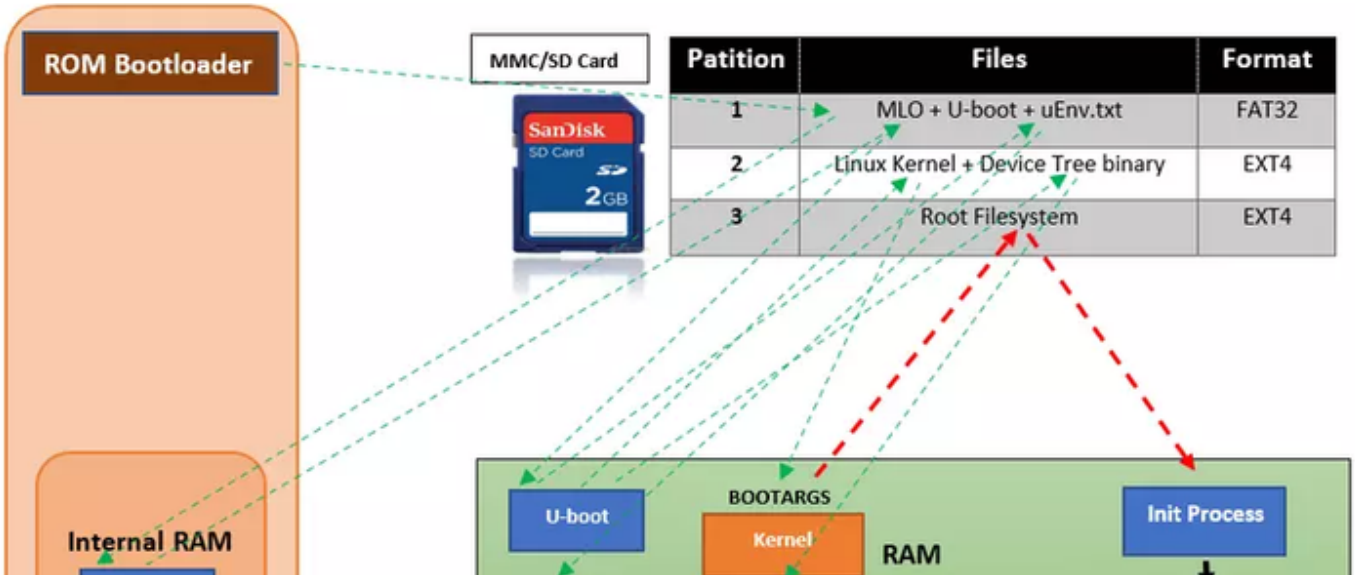


Next, U-Boot loads the kernel and device tree into RAM at predefined addresses, configured either in the U-Boot source code or in uEnv.txt. Finally, it passes the kernel parameters and transfers execution to the kernel.



Linux Kernel

After receiving control and the kernel parameters from U-Boot, the kernel mounts the root filesystem (rootfs) and runs the init process. Init is the first process started when the system boots successfully, and it runs until the system shuts down. Init starts other user-space processes, including applications that interact directly with the user. At this point, the system is ready for use.



alt text

Conclusion

The boot process of an embedded system can be divided into several stages:

- **ROM code:** Boot code written by the manufacturer that users cannot change. Its main job is to prepare the system to load the SPL into internal RAM.
- **SPL:** The secondary program loader. It initializes the required components and loads U-Boot into RAM.
- **U-Boot:** Loads the operating-system components (kernel, device tree, and, where applicable, information needed to locate the root filesystem) into memory, passes the kernel parameters, and transfers control to the kernel.
- **Linux kernel:** Mounts the root filesystem (rootfs) and runs the init process.

2. Kernel Image

2.1 Fundamental concepts

- **Kernel:** A program, and the most special one in the system: it starts early, runs continuously until shutdown, and manages all other programs.
- **Image:** In this context, a sequence of bytes packaged to be placed into memory as represented by the file. A kernel image is a file containing the kernel's machine code and data, ready to be loaded into RAM and executed.
- **Machine code:** Binary instructions that the CPU executes directly. C source code must be compiled to become machine code.

2.2 What's inside a kernel image?

Section	Contents
`.text`	Kernel machine code (scheduler, memory management, filesystems, network stack, built-in drivers)
`.rodata`	Read-only data, such as constant strings and `of_match_table` tables
`.data`	Initialized global variables
`.bss`	Zero-initialized global variables. The file stores the size, not the contents; the kernel clears this area to zero at boot

Section	Contents
`.init.*`	Code and data used only during boot. After boot completes, the kernel frees this area to reclaim RAM

2.3 Role of the kernel image in the system

When the kernel image runs, the kernel is responsible for:

- **Running in privileged mode:** This is the CPU privilege level that allows direct access to hardware. User programs run at a restricted privilege level. These two domains are called kernel space and user space.
- **Mediating between software and hardware:** User programs generally cannot access hardware directly; they call the kernel through system calls. A system call is the mechanism by which a user program asks the kernel to perform an operation, such as open, read, write, or fork.
- **Managing resources:**
  - **Processes:** Creation, scheduling, and termination.
  - **Memory:** RAM allocation, virtual memory (which gives each process its own address space), and protection between processes.
  - **Devices:** Controlling hardware through drivers.
  - **Files:** Managing filesystems.
  - **Networking:** Managing TCP/IP protocols and sockets.

3. Device Tree

What is a device tree?

A device tree is a data structure that describes a board's hardware. It is written as text for the kernel to interpret, rather than having the hardware information hard-coded into the kernel source.

What problem does the device tree solve?

- **Kernel:** The core of the operating system, responsible for managing the CPU, memory, and hardware.
- **Driver:** Code that allows the kernel to control a particular hardware device, such as UART, I2C, or GPIO.
- On the ARM architecture, hardware does not automatically tell the kernel which devices exist or at which addresses they are located. Unlike PCs, which commonly use PCI/ACPI for device discovery, the kernel cannot independently determine, for example, the register address of a board's UART or which interrupt number it uses.

Previously, this information was written directly into kernel code for each board, in what were called board files. Every new board required changes to the kernel and a separate kernel build. This made the kernel source larger and harder to maintain.

Structure

A device tree is a tree made up of:

- **Node:** Represents a device or a bus (a path used to transfer data). A node can contain child nodes.
- **Property:** A name-value pair inside a node that describes a characteristic of the device.

```
/ {
    soc {
        uart0: serial@40001000 {
            compatible = "vendor,uart-v1";
            reg = <0x40001000 0x400>;
            interrupts = <12>;
            status = "okay";
        };
    };
};
```

Common properties:

- `compatible`: A string identifying the device type. The kernel uses it to select a matching driver.
- `reg`: The start address and size of the device's register region.
- `interrupts`: The interrupt number used by the device to notify the CPU.
- `status`: `"okay"` enables the device; `"disabled"` disables it.

Workflow

- **DTS (Device Tree Source):** The human-readable text file, with the `.dts` extension (`.dtsi` is used for shared parts).
- **DTC (Device Tree Compiler):** The tool that compiles DTS into binary form.
- **DTB (Device Tree Blob):** The compiled binary `.dtb` file.
- The bootloader (the first boot-stage program that runs after the earlier ROM code, such as U-Boot) loads the kernel and DTB into RAM, then passes the DTB's address to the kernel.
- The kernel parses the DTB and creates the corresponding devices. It then matches each node's compatible property against the compatible identifiers declared by drivers (usually

in an of_match_table). When a match is found, the driver's probe() function is called to initialize the device.

4. Root Filesystem (rootfs)

4.1 What is a rootfs?

Rootfs, short for root filesystem, is the filesystem mounted at the root / of the directory tree. Every other file and directory in the system resides within it or is mounted beneath it.

4.2 Fundamental concepts

- **Filesystem:** A way to organize data on a storage device into files and directories so that the kernel and programs can locate, read, and write it. Examples include ext4, squashfs>
- **Mount:** The operation by which the kernel attaches a filesystem to a location in the directory tree, called a mount point, so its contents can be accessed by path.
- **Root (`/`):** The top-level directory of the Linux directory tree. The rootfs is the filesystem mounted at `/`.

The kernel image contains the kernel program itself. The rootfs contains the other components the system needs to run: user programs, libraries, configuration files, and device files.

4.3 What a root filesystem commonly contains

Directory	Contents
`/bin`, `/sbin`	Basic executables (shell, `ls`, `init`, etc.)
`/lib`	Shared libraries, such as libc
`/etc`	System configuration files
`/dev`	Device files representing hardware, such as `/dev/ttyS0`
`/proc`, `/sys`	Virtual filesystems created by the kernel that expose kernel and device information. Their contents are not stored on disk as ordinary files
`/lib/modules`	Driver modules (`.ko` files)

Directory	Contents
`/usr`, `/var`, `/home`, `/tmp`	Additional programs, variable data, user directories, and temporary files

5. Init Startup

5.1 What is init?

After the kernel initializes the required components and successfully mounts the root filesystem, it starts the first program in user space.

This program is usually `init`, with PID 1.

Init is responsible for setting up the user-space environment and managing system services and processes.

5.2 What happens during init startup?

The user-space startup process typically includes:

- The kernel mounts the root filesystem.
- The kernel starts the `init` program, usually with PID 1.
- Init runs startup scripts and configures the system.
- Required services are started, such as networking, logging, and embedded applications.
- The system becomes ready to perform its main functions.

6. Complete Embedded Linux Boot Flow

```
Power On / Reset
       |
       v
   Boot ROM
       |
       v
```

```
  SPL (if required)
       |
       v
     U-Boot
       |
       +------------------+
       |                  |
       v                  v
  Kernel Image            DTB
       |                  |
       +--------+---------+
                |
                v
          Linux Kernel
                |
                v
     Mount Root Filesystem
                |
                v
           Init (PID 1)
                |
                v
   System Services / Applications
                |
                v
           System Ready
```

Summary of responsibilities:

- **Boot ROM:** Starts the boot process and loads the next boot stage.
- **SPL:** Initializes the hardware needed to load the full bootloader, if the platform uses an SPL.
- **U-Boot:** Loads the kernel and DTB, sets boot parameters, and transfers control to the kernel.
- **Kernel image:** Contains the Linux kernel code.
- **Device tree:** Provides the kernel with a description of the hardware.
- **Root filesystem:** Provides the programs, libraries, and files required in user space.
- **Init:** Initializes user space and manages system services.

Revision #1
Created 2026-10-23 09:49:59 UTC by Dat Nguyen
Updated 2026-10-23 10:16:16 UTC by Dat Nguyen